

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Бэк-энд разработка

Отчет

Практическая/Лабораторная работа

Выполнил:

Цатинян Артём

Группа К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

## **Задача**

Тема лабораторного проекта:

Приложение для бронирования столиков в ресторанах. В рамках домашнего задания требовалось реализовать тестирование API средствами Postman и написать тесты внутри Postman. В результате выполнения работы необходимо было подготовить один комплексный сценарий по основному процессу приложения.

## **Ход работы**

Для выполнения задания была создана коллекция запросов в Postman для тестирования REST API приложения бронирования столиков в ресторанах. В качестве основного сценария был выбран пользовательский процесс от регистрации до создания и отмены бронирования.

Для запуска сценария было подготовлено локальное окружение Postman с переменными:

- baseUrl
- email
- password
- firstName
- lastName
- phone
- accessToken
- restaurantId
- tableId
- bookingId
- startsAt
- endsAt

В качестве базового адреса API использовался локально запущенный backend-сервис по адресу <http://localhost:8080/api/v1>. В коллекции Postman были реализованы следующие запросы:

### **1. Регистрация пользователя**

Был создан запрос `POST /auth/register`, который позволяет зарегистрировать нового пользователя в системе. В тестах проверялся успешный HTTP-статус ответа, наличие JWT-токена в ответе, а также сохранение токена в переменную окружения.

## **2. Авторизация пользователя**

Был создан запрос `POST /auth/login`. В тестах проверялось, что сервер возвращает корректный токен доступа, который затем использовался для выполнения защищенных запросов.

## **3. Получение профиля текущего пользователя**

Для проверки авторизации использовался запрос `GET /users/me`. В тестах сравнивался email пользователя из ответа с email, указанным при регистрации.

## **4. Получение списка кухонь**

Был создан запрос `GET /cuisines` с параметрами поиска и сортировки. Этот шаг позволял продемонстрировать работу справочника кухонь и показать источник данных для дальнейшей фильтрации ресторанов.

## **5. Получение списка ресторанов с фильтрацией, сортировкой и пагинацией**

Использовался запрос `GET /restaurants` с query-параметрами `city`, `cuisine`, `priceSegment`, `guestsCount`, `page`, `size`, `sortBy`, `sortDir`. В тестах проверялось, что список ресторанов не пуст, а идентификатор первого ресторана сохраняется в переменную окружения для последующих запросов.

## **6. Получение информации о ресторане**

Был реализован запрос `GET /restaurants/{restaurantId}`. На этом шаге проверялась корректность получения детальной информации о выбранном ресторане.

## **7. Получение меню ресторана с фильтрацией**

Использовался запрос ``GET /restaurants/{restaurantId}/menu`` с параметрами фильтрации по категории и диапазону цены. В тестах проверялось, что в ответе присутствуют категории меню и позиции блюд.

## **8. Получение доступных слотов бронирования**

Для поиска доступного времени использовался запрос ``GET /restaurants/{restaurantId}/availability`` с параметрами даты и количества гостей. Из ответа извлекались ``tableId``, ``startsAt`` и ``endsAt``, которые затем использовались в создании бронирования.

## **9. Создание бронирования**

Был создан запрос ``POST /bookings``. В теле запроса передавались идентификаторы ресторана и столика, временной интервал бронирования, количество гостей и специальные пожелания. В тестах проверялось, что бронирование успешно создается, получает идентификатор и статус ``CONFIRMED``.

## **10. Получение списка бронирований пользователя**

Для проверки результата использовался запрос ``GET /bookings/me`` с пагинацией и сортировкой. В тестах проверялось, что только что созданное бронирование присутствует в списке.

## **11. Получение бронирования по идентификатору**

Был реализован запрос ``GET /bookings/{bookingId}``. В тестах проверялось, что сервер возвращает корректную запись, соответствующую сохраненному ``bookingId``.

## **12. Отмена бронирования**

Завершающим шагом сценария был запрос ``PATCH /bookings/{bookingId}/cancel``. В тестах проверялось, что статус бронирования изменяется на ``CANCELLED``. Во всех запросах были добавлены Postman-тесты с использованием ``pm.test(...)``. Основные проверки включали:

- проверку HTTP-статуса ответа;
- проверку обязательных полей JSON-ответа;

- сохранение значений из ответа в переменные окружения;
- проверку связности сценария между запросами.

Подготовленный сценарий охватывает основной пользовательский процесс в приложении и демонстрирует работу как публичных, так и защищенных эндпоинтов. Кроме того, в сценарии были показаны фильтрация, сортировка и пагинация, что важно для полноты тестирования API.

## **Вывод**

В результате выполнения практической работы было реализовано тестирование REST API приложения бронирования столиков в ресторанах средствами Postman. Была подготовлена коллекция запросов с единым комплексным пользовательским сценарием: регистрация, авторизация, поиск ресторана, просмотр его меню, получение доступных слотов, создание бронирования, просмотр и отмена бронирования. Для каждого шага были добавлены автоматические проверки внутри Postman, что подтверждает корректность работы основных методов API.